Additional Questions and Answers related to Lecture 6

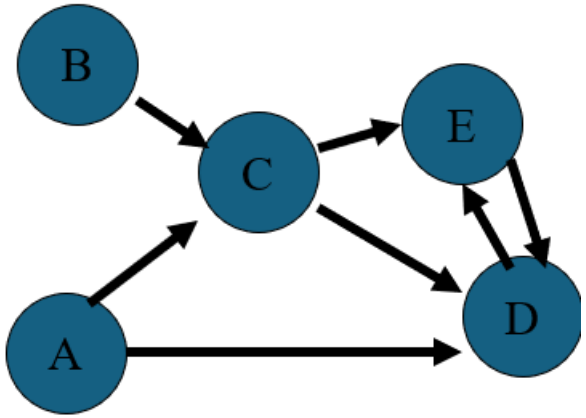
M.G.Noel A.S Fernando (PhD)

Professor

NSBM/Dept. of Information Systems Engineering, UCSC

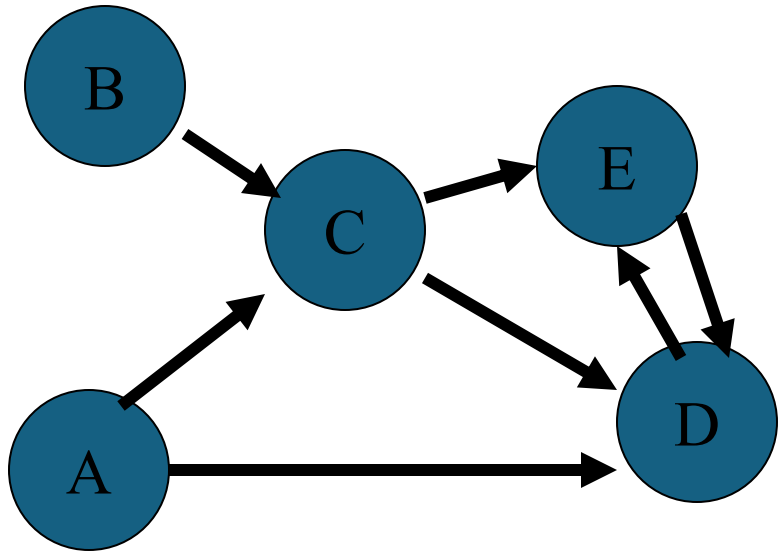
Question 1

- Consider the following directed graph.



- Find the Adjacency matrices (adj, Adj_2,Adj_3,Adj_4 and Adj_5)
- Find the path matrix (transitive closure)

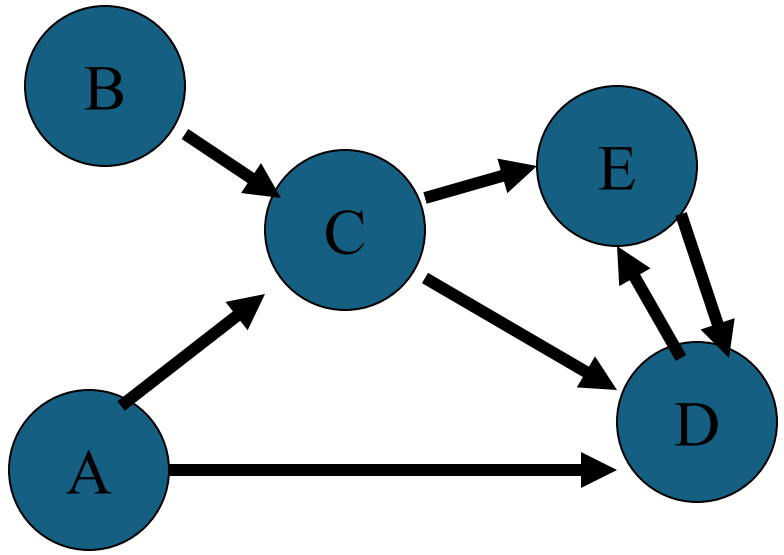
Lets compute the matrix mortification of adj with itself, with numerical mortification replaced by conjunction (the AND operation) and addition is replaced by disjunction (the OR operation)



Adj_2

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	0
E	0	0	0	0	1

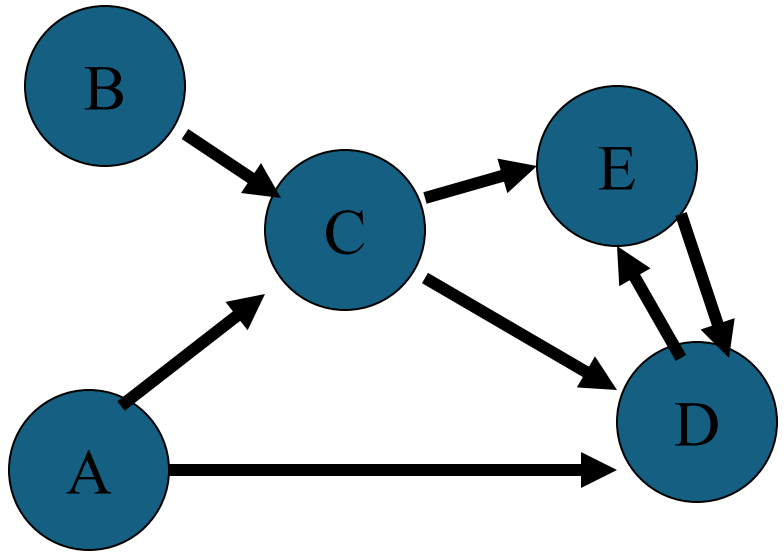
Adj_3



Adj_3

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	0	1
E	0	0	0	1	0

ADj_4



Adj_4

	A	**B**	**C**	**D**	**E**
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	0
E	0	0	0	0	1

Path matrix

- Assume that we want to know whether a path of length 4 or less 4 exists between two nodes of a graph.
- If such a path exists between node i and j, it must be of length 1, 2, 3 or 4. If there is a path of length 4 or less between node i and node j , the value of adj[i][j] or adj_2[i][j] or adj_3[i][j] or adj_4[i][j] must be true.

Path matrix

- Suppose we wish to construct a path matrix such that path[i][j] equals true if and only if there is some path from node i to node j.
- Path[I][j]=adj[I][j] or adj_2[I][j] or adj_3[I][j]or adj_4[I][j]

- If the graph has n nodes, it must be true that

Path [j][j] =adj[i][j] or

adj_2[i][j] or

adj_2[i][j] or

adj_3[i][j] or

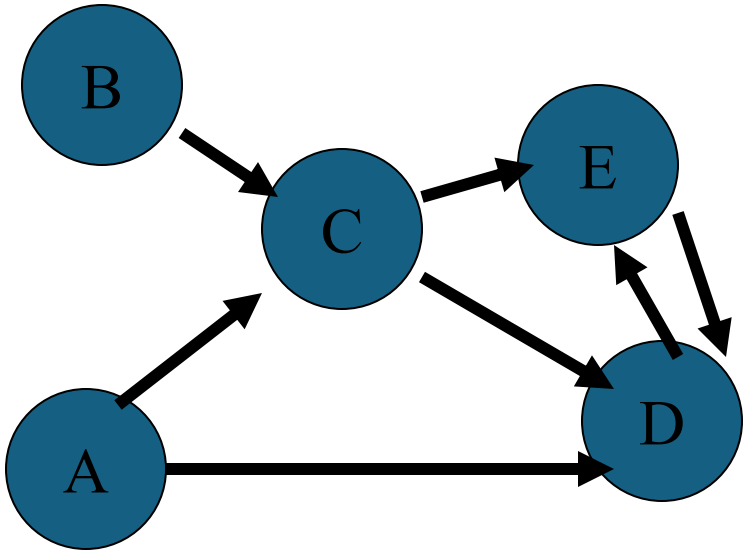
.....

adj_n[i][j]

Path[i][j]= 1 if and only if there is some path of any length from node i to node j

Path[i][j]= 0 otherwise

The matrix path is called the ***transitive closure*** of the matrix adjacency



	A	B	C	D	E
A	0	0	1	1	0
B	0	0	1	0	0
C	0	0	0	1	1
D	0	0	0	1	1
E	0	0	0	1	0

ADj

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	0	1
E	0	0	0	0	0

ADj_2

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	0	1
E	0	0	0	1	0

ADj_3

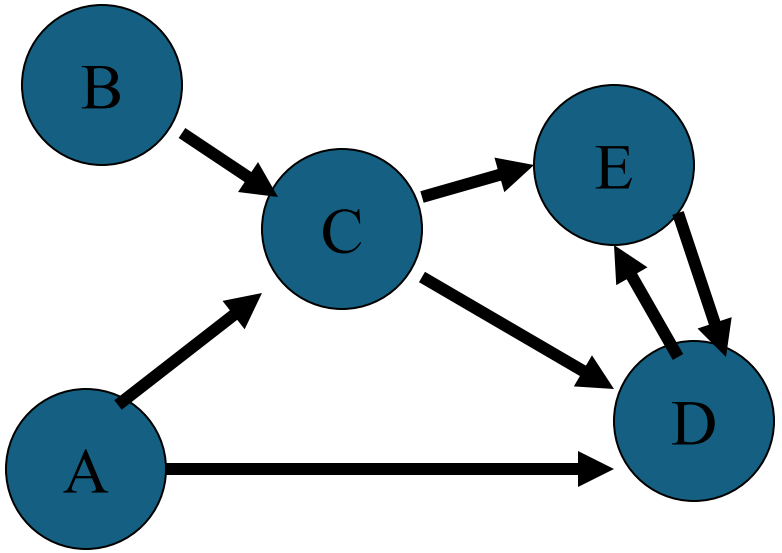
	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	0
E	0	0	0	0	1

ADj_4

	A	B	C	D	E
A	0	0	0	1	1
B	0	0	0	1	1
C	0	0	0	1	1
D	0	0	0	1	0
E	0	0	0	0	1

ADj_5

•Path[i][j]=adj[i][j] or adj_2[i][j] or adj_3[i][j]or adj_4[i][j] or adj_5[i][j]



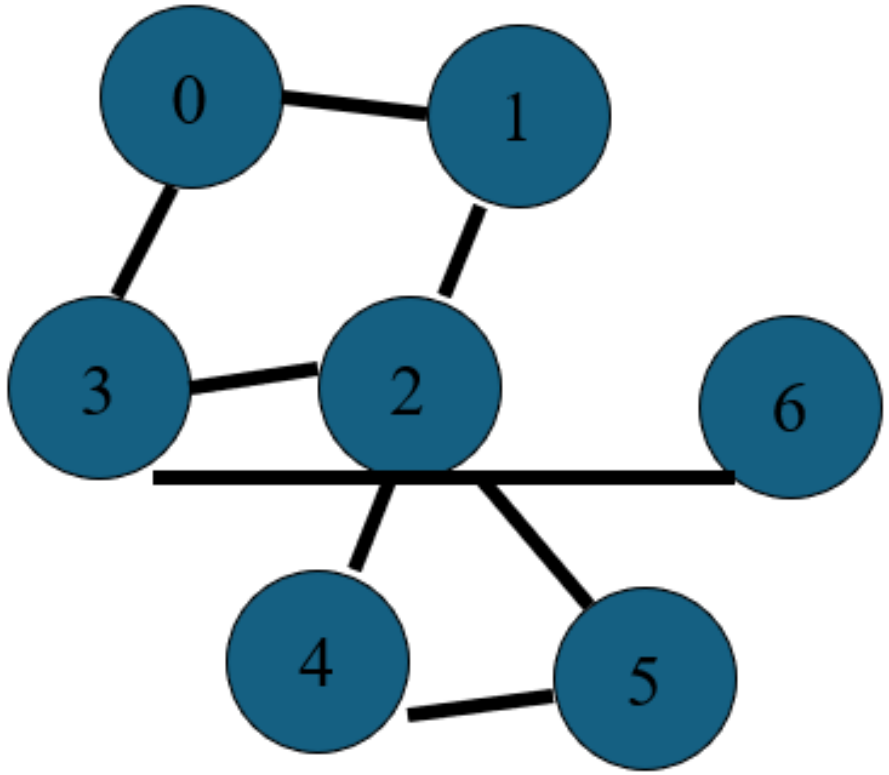
	A	B	C	D	E
A	0	0	1	1	1
B	0	0	1	1	1
C	0	0	0	1	1
D	0	0	0	1	1
E	0	0	0	1	1

path

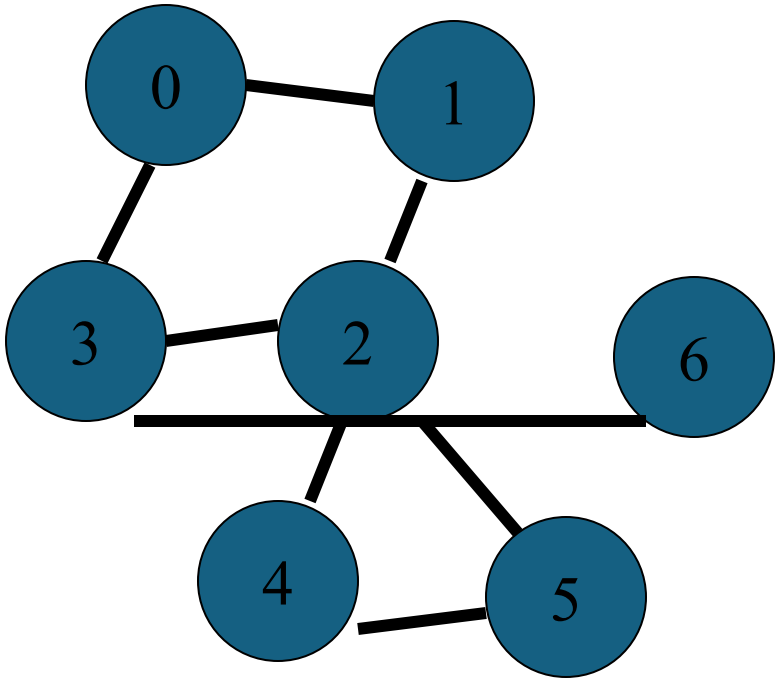
Ex : C Codes ? [adj and path matrix for the path of length is equal to n

Question 2

- Consider the following Graph.
- Creating the edge table and traversing it using Depth First Traversal.



Answer (DFT/DFS)



The edge table for the above graph.

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

To do a depth- first traversal of this graph, first we choose a starting vertex (say 0)

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

Let 0 be the starting vertex.

Can select {1,3 }

We move to vertex 1, vertex 1 is connected to vertices 0 and 2, however vertex 0 has already been visited. And move along to vertex 2.

The depth first traversal has so far visited vertices in the order

0→ 1 → 2

The edge table after vertices 0, 1 and 2 have been visited.

(0→ 1→ 2, it is important to keep track of the order in which the vertices are visited, since we may need to back track later on)

To continue, (from 2),

Can select {3, 4 , 5}

If we choose 3 , this leads us to vertex 6,

So for visited nodes are

0→1→2→3→6

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

If we visit the row corresponding to vertex 6. There are no Un-visited vertices listed therefore, we must backtrack until the last row already examine that has an un-visited vertex listed on it.We back track by going backwards one vertex at a time in the list until we come to a vertex that has some un-visited edges arising from it.

Backing up from 6->3 we see from the edge table that all its vertices have been visited.

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

So back up 3→2 Here first un-visited vertex is 4, then we can visit 4→2→4→5

The final traversal is

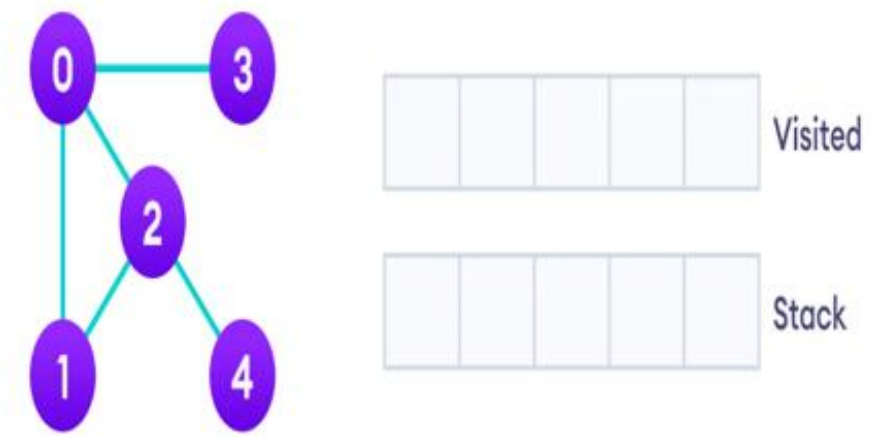
0→1→2→-3→6
→4→5

Note : the depth traversals of a graph are not unique. If we had listed the vertices in each row of the edge table in a different order, we would get a different traversal.

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

Question 3

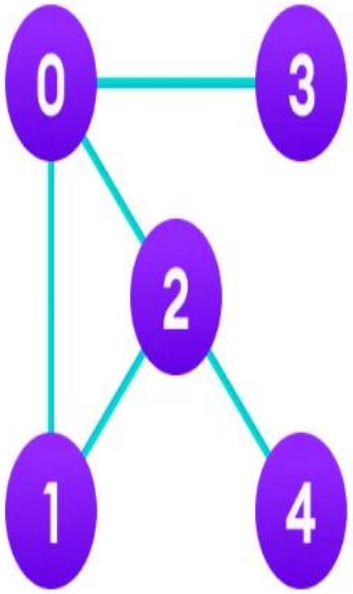
- Consider the following Graph.



- Explain how to traverse the above graph using Depth First Traversal with a stack.
- Write the pseudocode algorithm for DFT

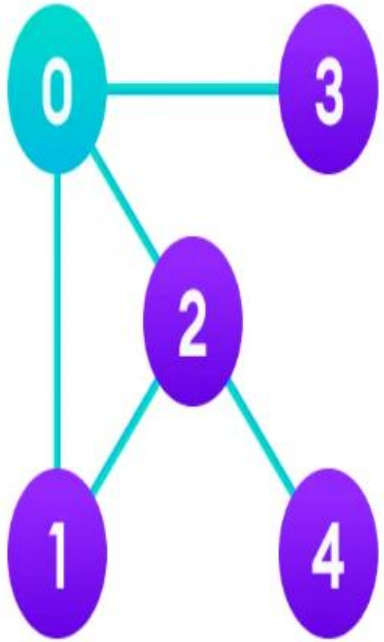
Implementation of Depth First Search/ Traversal using a Stack

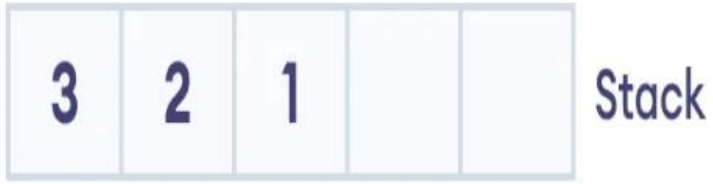
- **Depth First Search Example**
- We use an undirected graph with 5 vertices.



Implementation of Depth First Search/Traversal using a Stack

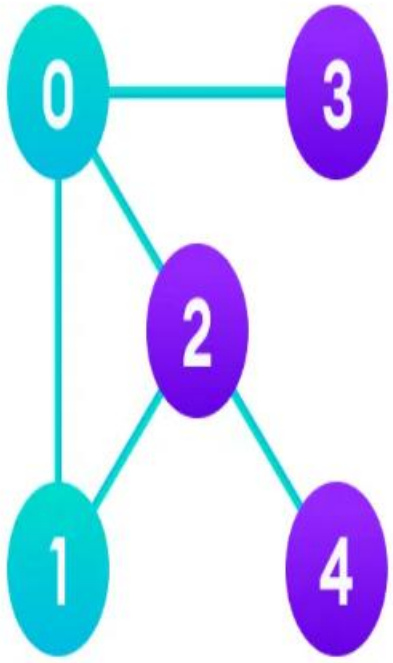
- We start from vertex 0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.

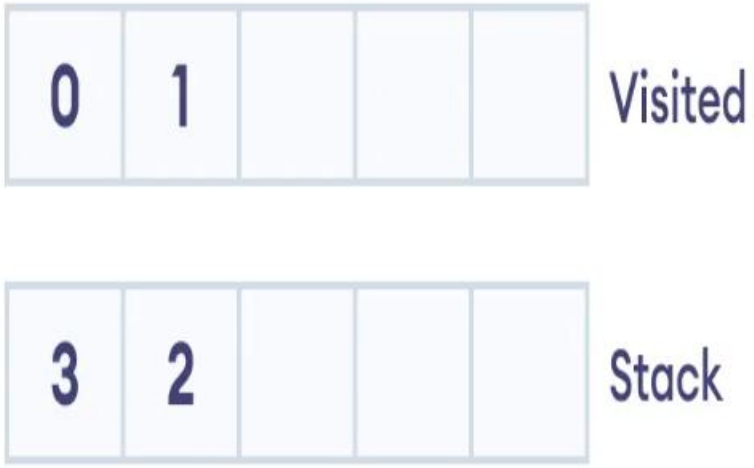




Implementation of Depth First Search/ Traversal using a Stack

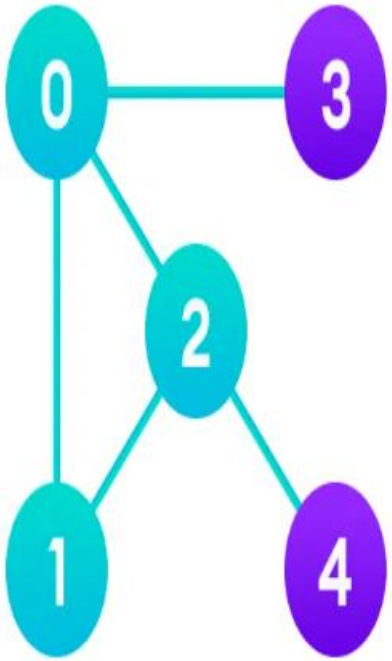
- Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

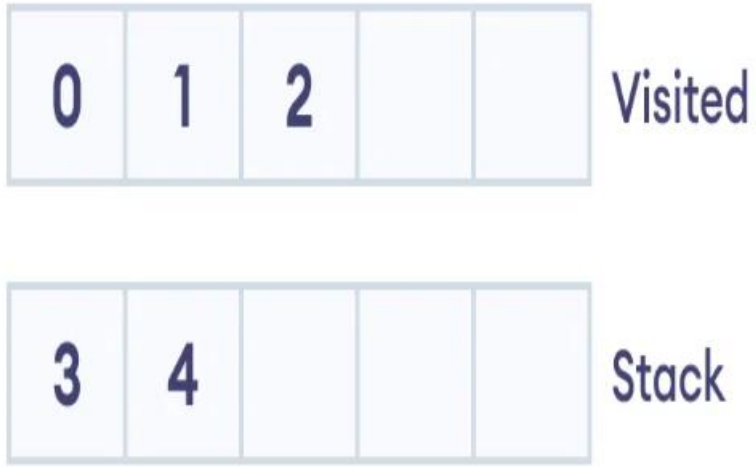




Implementation of Depth First Search/ Traversal using a Stack

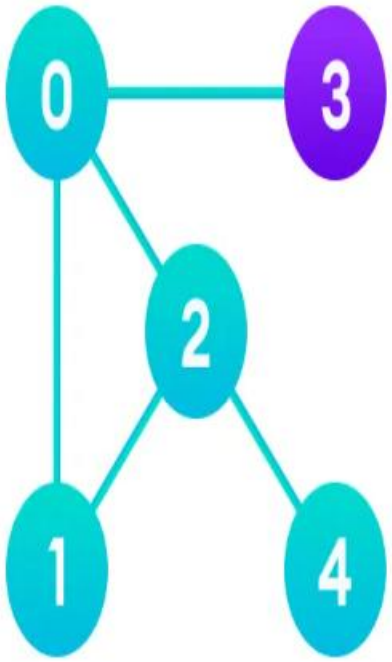
- Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.

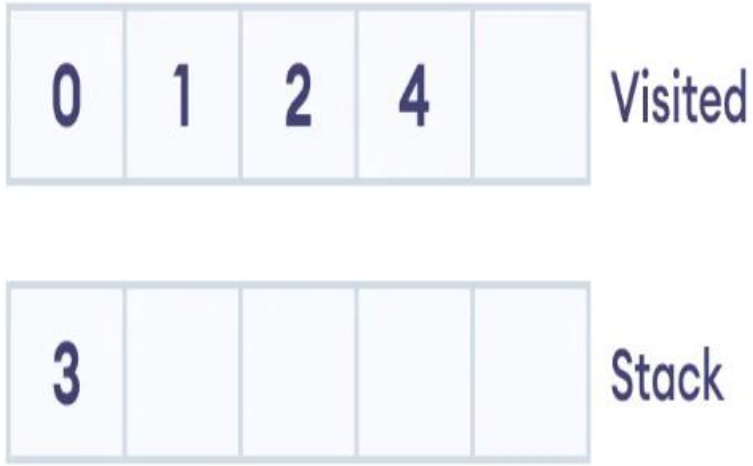




Implementation of Depth First Search/Traversal using a Stack

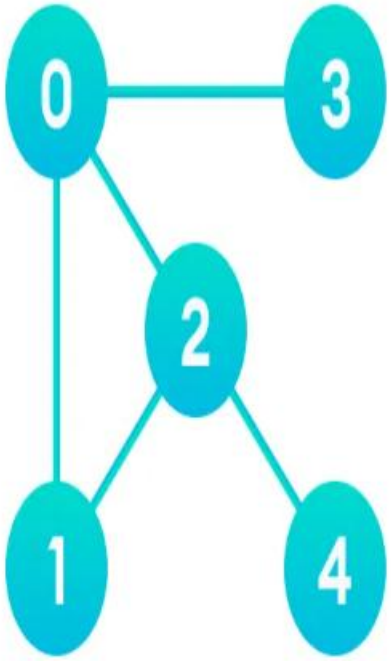
- Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit

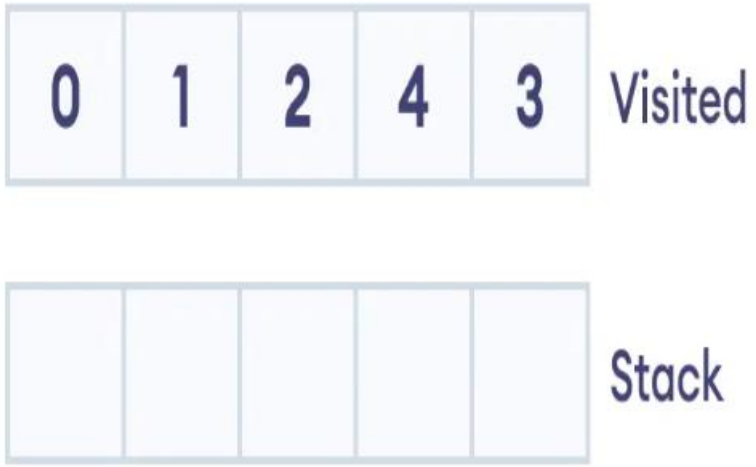




Implementation of Depth First Search/Traversal using a Stack

- After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.





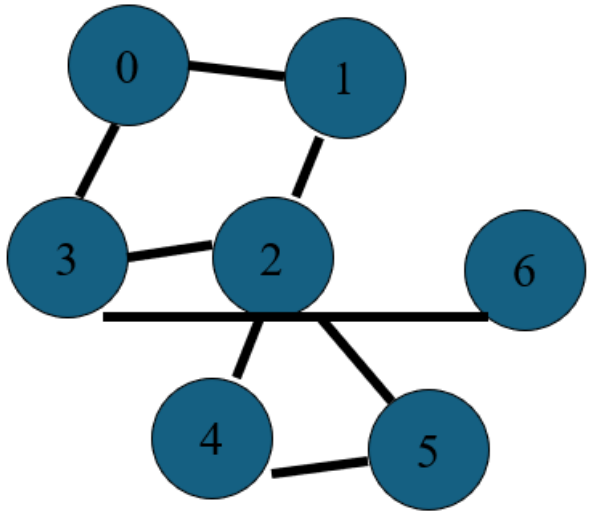
DFS Pseudocode (recursive implementation)

```
DFS(G, u)
     u.visited = true
     for each v ∈ G.Adj[u]
          if v.visited == false
               DFS(G,v)

init() {
     For each u ∈ G
          u.visited = false
      For each u ∈ G
        DFS(G, u)
}
```

Question 4

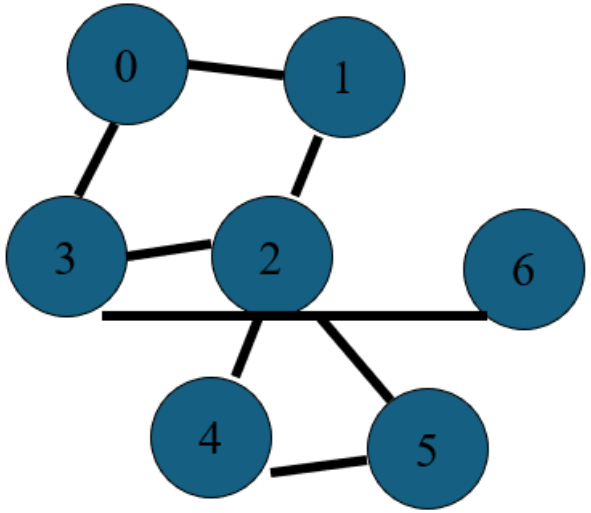
- Consider the following graph.



- Creating the edge table and traversing it using Breadth First Traversal.

Breadth First Traversal

- First, we choose a starting vertex (say 0)
- The easiest way to handle this is to use a queue.
- Each time we add a vertex to the traversal, we add to the queue all visited vertices listed in the row corresponding to the added vertex. We then add the vertex at the traversal and add any un-visited vertices in its row to the tail of the queue.
- We continue until all the vertices in the graph have been added to the traversal, at which point the queue should be empty.



First we choose a starting vertex (say 0)

Adding vertex 0 the the traversal.

We add 1 and 3 to the queue.

Then vertex 1 is at the head of the queue, so it is next vertex to be added to the traversal.

Looking at row in the edge table, we see that the only un-visited , in that row is vertex 2, so it is added to the queue.

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

The vertex at the head head of the queue is now vertex 3, so it is added to the traversal and the only un-visited vertex in 3 is 6 and added to the queue.

At this point

The traversal contains the vertices 2 and 6

At this point edge table is

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

We now continue by removing vertex 2 from the queue adding to the traversal.

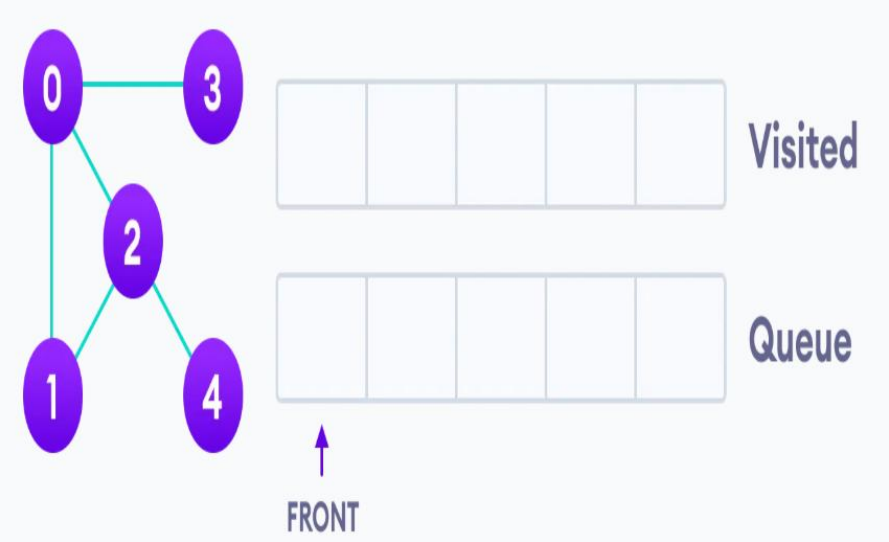
The final traversal order is

0 1 3 2 6 4 5

Node	Vertex list
0	1 3
1	0 2
2	1 3 4 5
3	0 2 6
4	2 5
5	2 4
6	3

Question 5

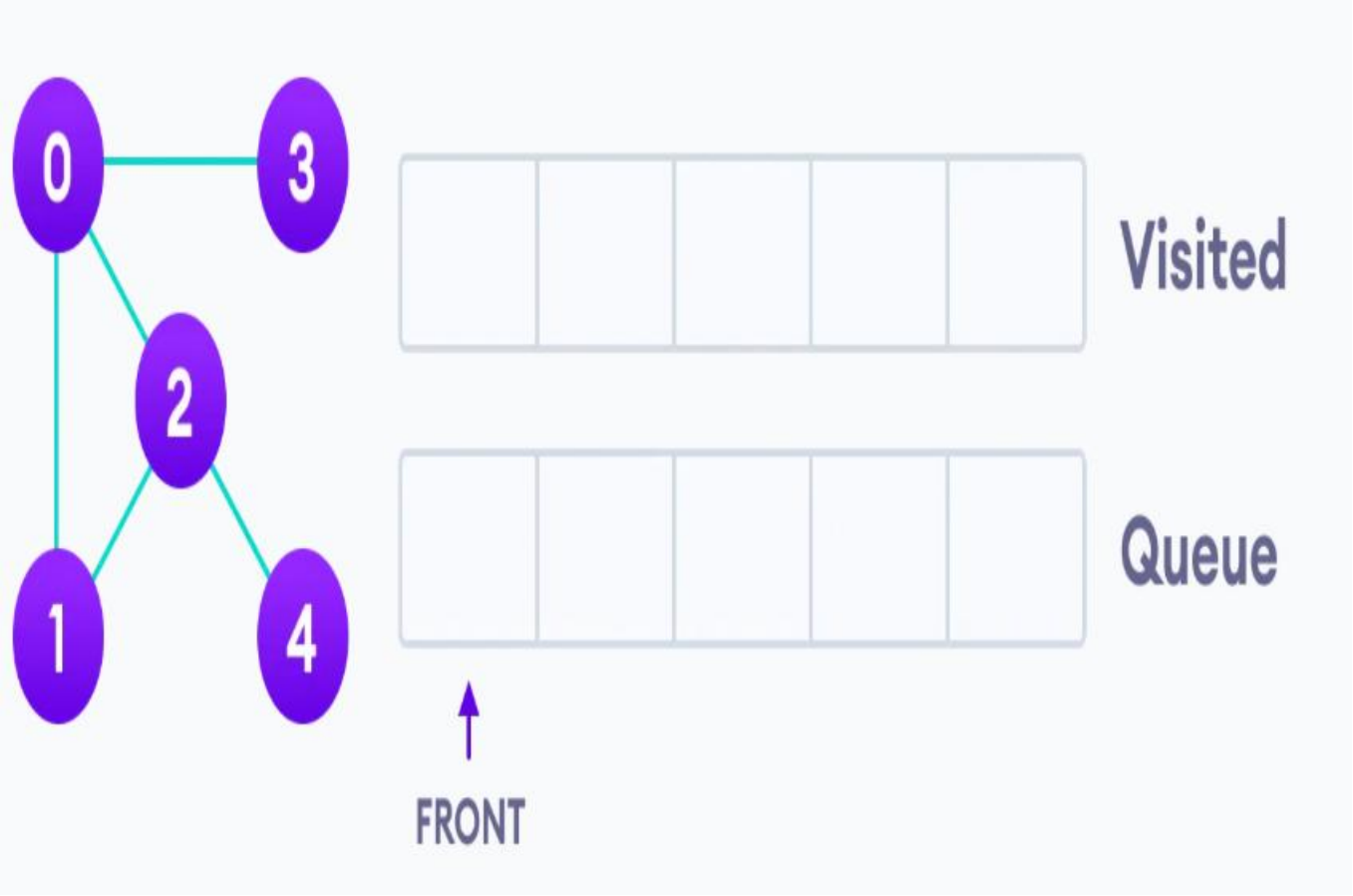
- Consider the following Graph.



- Explain how to traverse the above graph using Breadth First Traversal with a queue.
- Write the pseudocode algorithm for BFT/BFS

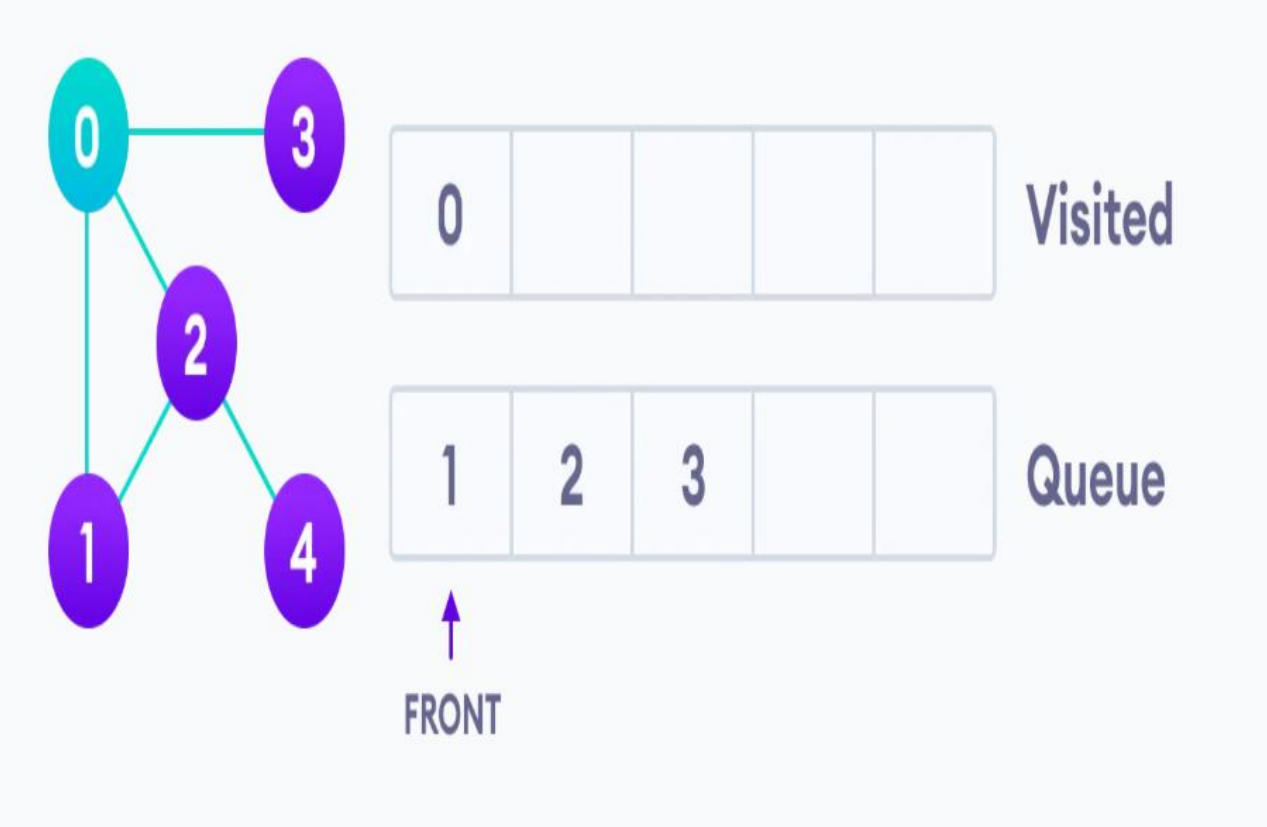
Implementation of **BFS** Search/Traversal using a Queue

- **BFS example**



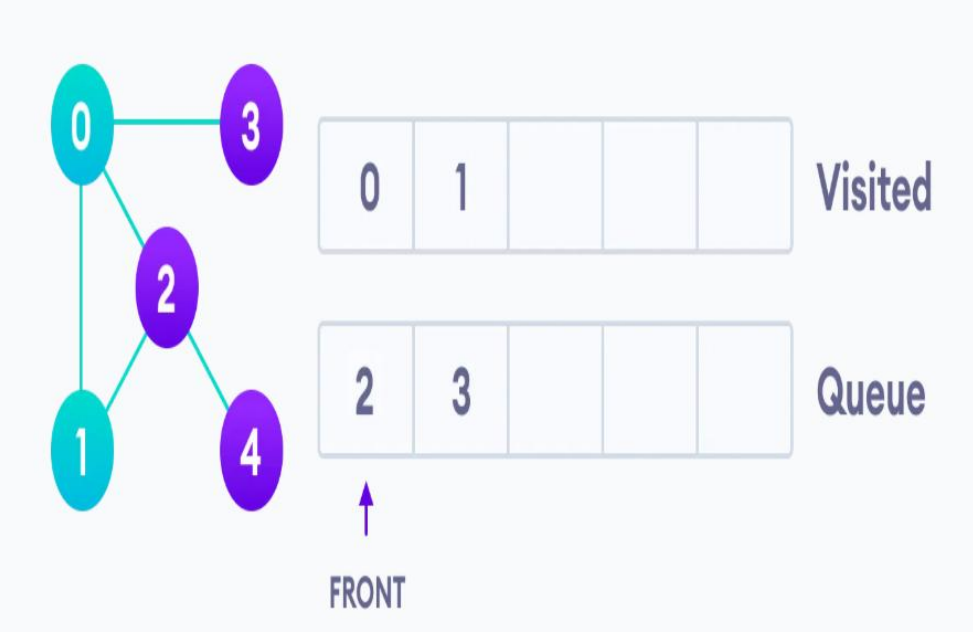
Implementation of **BFS** Search/Traversal using a Queue (Cont..)

- We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the queue.



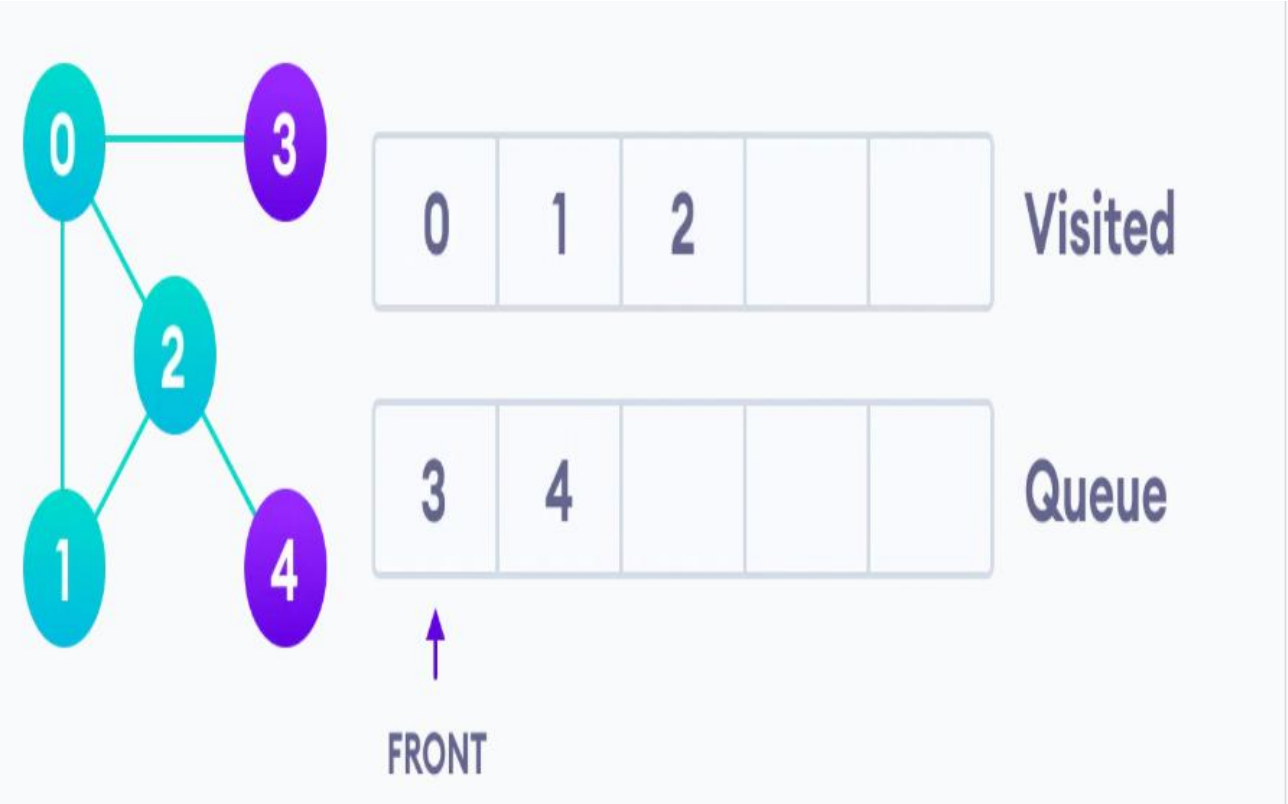
Implementation of **BFS** Search/Traversal using a Queue (Cont..)

- Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.



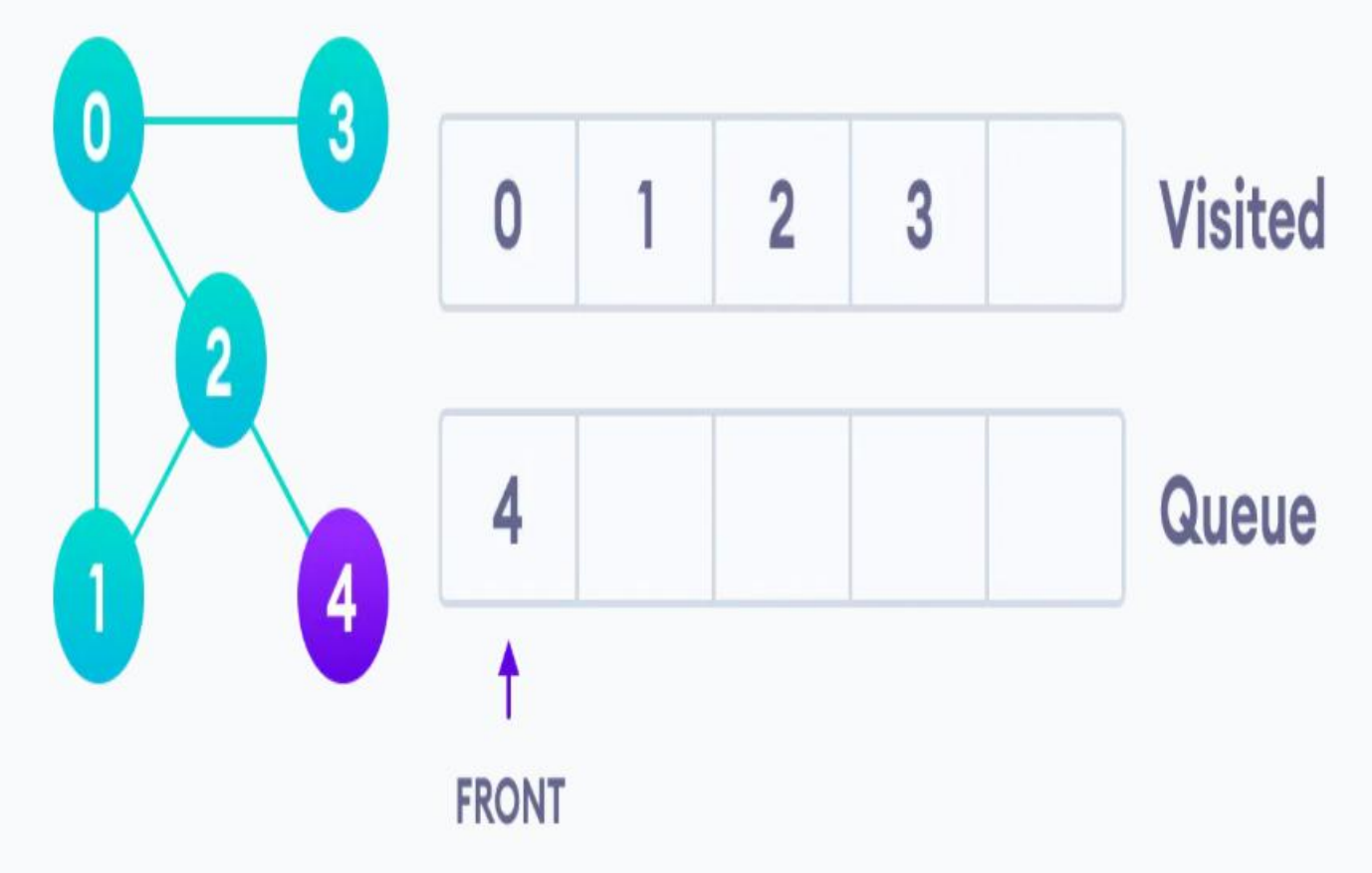
Implementation of **BFS** Search/Traversal using a Queue (Cont..)

- Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.



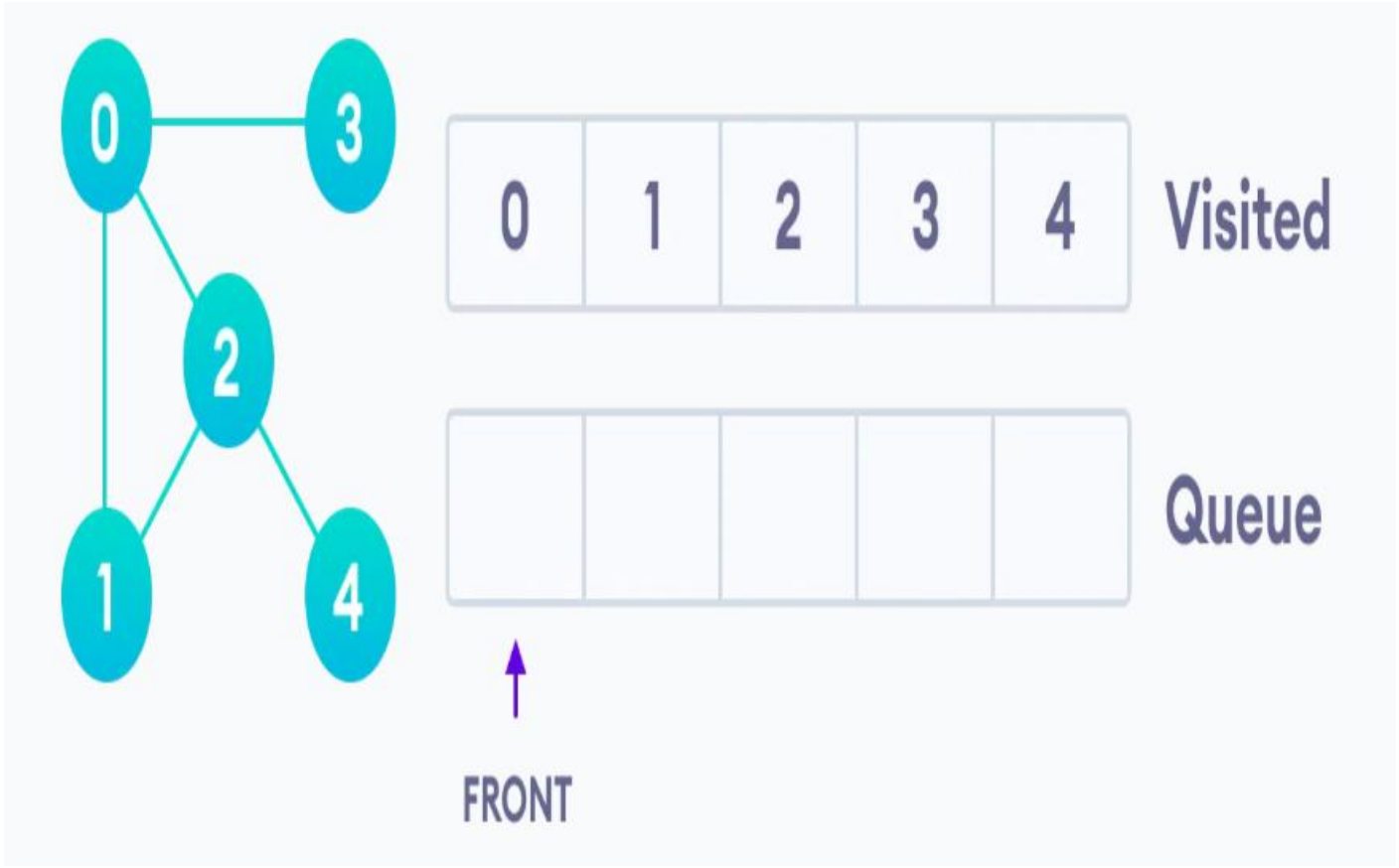
Implementation of **BFS** Search/Traversal using a Queue (Cont..)

- Visit 2 which was added to queue earlier to add its neighbors



Implementation of **BFS** Search/Traversal using a Queue (Cont..)

- Only 4 remains in the queue since the only adjacent node of 3 i.e. 0 is already visited. We visit it.



BFS pseudocode

```
create a queue Q
mark v as visited and put v into Q
while Q is non-empty
    remove the head u of Q
    mark and enqueue all (unvisited) neighbours of u
```